



Banco de Preguntas — Parcial 2

Guía de estudio para comprender el funcionamiento de la plataforma (`testcinar26`) y los conceptos del curso. No se trata de memorizar, sino de entender cómo y por qué funciona.

Tema 1 — Navegación en la aplicación

Pregunta 1

SELECCIÓN MÚLTIPLE

¿Qué tipo de enrutamiento utiliza SvelteKit para definir las rutas de la aplicación?

- a Enrutamiento basado en configuración JSON
- b Enrutamiento basado en el sistema de archivos (filesystem-based routing) ✓
- c Enrutamiento manual con un router global
- d Enrutamiento mediante anotaciones decoradoras

Respuesta: b) Enrutamiento basado en el sistema de archivos (filesystem-based routing)

Justificación: SvelteKit usa enrutamiento basado en el sistema de archivos (filesystem-based routing): cada carpeta o archivo dentro de `src/routes/` se convierte automáticamente en una URL. En este proyecto, `src/routes/dashboard/+page.svelte` responde en `/dashboard`.

Pregunta 2

SELECCIÓN MÚLTIPLE

¿Qué archivo define una página navegable dentro de una ruta en SvelteKit?

- a `page.svelte`
- b `+page.svelte` ✓
- c `index.svelte`
- d `route.svelte`

Respuesta: b) `+page.svelte`

Justificación: `+page.svelte` es el archivo especial (prefijo `+`) que define una página navegable. Por ejemplo, `src/routes/desarrollo-web-1/parcial-1/+page.svelte` es la página del Parcial 1.

Pregunta 3

SELECCIÓN MÚLTIPLE

¿Qué archivo envuelve a las páginas hijas con una estructura común (header, footer, etc.)?

a `+layout.svelte` ✓

b `+page.svelte`

c `+error.svelte`

d `+server.ts`

Respuesta: a) `+layout.svelte`

Justificación: `+layout.svelte` envuelve a las páginas hijas con una estructura común. En este proyecto, `src/routes/+layout.svelte` incluye el Header global y renderiza el contenido con `{@render children()}`.

Pregunta 4

SELECCIÓN MÚLTIPLE

¿Cómo se realiza navegación programática en SvelteKit?

a `window.location = "/ruta"`

b `navigate("/ruta")` desde `$app/router`

c `goto("/ruta")` desde `$app/navigation` ✓

d `redirect("/ruta")` en el template

Respuesta: c) `goto("/ruta")` desde `$app/navigation`

Justificación: `goto()` de `$app/navigation` es la forma programática de navegar. En `Header.svelte` se usa `goto("/login")` al cerrar sesión.

Pregunta 5

SELECCIÓN MÚLTIPLE

¿Para qué se usa `redirect(303, "/ruta")` en SvelteKit?

a Navegación en el cliente tras un clic

b Redirección desde las load functions (server-side) ✓

c Cargar una imagen de forma diferida

d Mostrar un error 404

Respuesta: b) Redirección desde las load functions (server-side)

Justificación: `redirect(303, "/ruta")` se usa dentro de las load functions para redirigir desde el servidor antes de renderizar la página.

Pregunta 6

SELECCIÓN MÚLTIPLE

Dada la estructura `src/routes/usuario/[id]/+page.svelte`, ¿qué URL resuelve?

a `/usuario/id`

b `/usuario/[id]`

c `/usuario/123` ✓

d `/123/usuario`

Respuesta: c) `/usuario/123`

Justificación: Los corchetes `[id]` definen una ruta dinámica: capturan el segmento de la URL y lo exponen como parámetro. Así, `usuario/[id]/+page.svelte` resuelve `/usuario/123`.

Tema 2 — Configuración de la página y SEO

Pregunta 7

SELECCIÓN MÚLTIPLE

¿Qué archivo es la plantilla HTML raíz de un proyecto SvelteKit?

a `index.html`

b `app.html` ✓

c `main.html`

d `document.html`

Respuesta: b) `app.html`

Justificación: `app.html` es la plantilla HTML raíz. Contiene los placeholders `%sveltekit.head%` y `%sveltekit.body%`; todo lo demás se inyecta allí.

Pregunta 8

SELECCIÓN MÚLTIPLE

¿Qué componente permite inyectar etiquetas en el `<head>` desde cualquier página?

- a `<svelte:meta>`
- b `<svelte:title>`
- c `<svelte:head>` ✓
- d `<head-tag>`

Respuesta: c) `<svelte:head>`

Justificación: `<svelte:head>` permite inyectar etiquetas en el `<head>` desde cualquier página. Cada página del proyecto (por ejemplo `parcial-1`) define su `<title>` de esta forma.

Pregunta 9

SELECCIÓN MÚLTIPLE

¿Qué placeholder de `app.html` es reemplazado por el contenido renderizado de la aplicación?

- a `%sveltekit.head%`
- b `%sveltekit.body%` ✓
- c `%sveltekit.assets%`
- d `%sveltekit.app%`

Respuesta: b) `%sveltekit.body%`

Justificación: `%sveltekit.body%` es donde SvelteKit renderiza la aplicación; `%sveltekit.head%` es donde se inyectan las metaetiquetas y el `<title>`.

Pregunta 10

SELECCIÓN MÚLTIPLE

¿Cuál es el propósito de la etiqueta `<meta name="description">` dentro de `<svelte:head>` ?

- a Cambiar el color de fondo de la página
- b Proveer una descripción que los buscadores muestran en los resultados (SEO) ✓
- c Cargar una hoja de estilos externa
- d Definir el idioma del documento

Respuesta: b) Proveer una descripción que los buscadores muestran en los resultados (SEO)

Justificación: La `<meta name="description">` es la descripción que los buscadores muestran en los resultados. Es la base del SEO on-page.

Pregunta 11

SELECCIÓN MÚLTIPLE

¿Cuál es la diferencia principal entre `+page.ts` y `+page.server.ts` ?

- a No hay diferencia, ambos son idénticos
- b `+page.ts` corre en el cliente (CSR) y `+page.server.ts` en el servidor (SSR) ✓
- c `+page.ts` solo sirve para estilos
- d `+page.server.ts` solo funciona con bases de datos

Respuesta: b) `+page.ts` corre en el cliente (CSR) y `+page.server.ts` en el servidor (SSR)

Justificación: `+page.ts` se ejecuta en el cliente (CSR), mientras que `+page.server.ts` se ejecuta en el servidor (SSR) y permite precargar datos antes de renderizar.

Pregunta 12

SELECCIÓN MÚLTIPLE

¿En qué archivo conviene ejecutar lógica que use secretos o tokens del servidor?

- a `+page.ts`
- b `+layout.ts`
- c `+page.server.ts` ✓
- d Cualquiera, es lo mismo

Respuesta: c) `+page.server.ts`

Justificación: Los secretos y tokens solo deben vivir en el servidor. `+page.server.ts` no se envía al navegador, a diferencia de `+page.ts`.

Pregunta 13

SELECCIÓN MÚLTIPLE

En Svelte 5, ¿qué rune se usa para declarar una variable reactiva?

a `$state()` ✓

b `$props()`

c `$derived()`

d `$effect()`

Respuesta: a) `$state()`

Justificación: `$state()` es el rune de Svelte 5 que declara variables profundamente reactivas; cuando cambian, el DOM se actualiza automáticamente.

Pregunta 14

SELECCIÓN MÚLTIPLE

¿Qué rune calcula un valor derivado que se recalcula cuando cambian sus dependencias?

a `$state()`

b `$derived()` ✓

c `$props()`

d `$effect()`

Respuesta: b) `$derived()`

Justificación: `$derived()` calcula un valor que se recalcula automáticamente cuando cambian sus dependencias. En `parcial-1/+page.svelte` se usa para `isUnlimited` y `slots`.

Pregunta 15

SELECCIÓN MÚLTIPLE

¿Cómo se reciben las propiedades (props) del componente padre en Svelte 5?

a `export let prop`

b `let prop = $props()`

c `let { prop } = $props()` ✓

d `@Prop prop`

Respuesta: c) `let { prop } = $props()`

Justificación: En Svelte 5 las props se reciben con `$props()`. `let { prop } = $props()` reemplaza al antiguo `export let` de Svelte 4.

Pregunta 16

SELECCIÓN MÚLTIPLE

¿Qué directiva crea un enlace bidireccional entre un input y una variable?

- a `on:input`
- b `bind:value` ✓
- c `model:value`
- d `sync:value`

Respuesta: b) `bind:value`

Justificación: `bind:value` crea un enlace bidireccional entre un input y una variable. En `login/+page.svelte` se usa para `username` y `password`.

Pregunta 17

SELECCIÓN MÚLTIPLE

¿Qué función de `svelte/store` crea un store global escribible?

- a `readable()`
- b `derived()`
- c `writable()` ✓
- d `store()`

Respuesta: c) `writable()`

Justificación: `writable()` de `svelte/store` crea estado global escribible. En `auth.js` se exportan `token` y `currentUser` como stores writable.

Pregunta 18

SELECCIÓN MÚLTIPLE

¿Cómo se accede al valor de un store dentro del template de Svelte?

- a Con paréntesis: `(miStore)`
- b Con el prefijo `$`: `{$miStore}` ✓
- c Con `await miStore`
- d Con `miStore.value`

Respuesta: b) Con el prefijo `$`: `{$miStore}`

Justificación: Dentro del template se accede con el prefijo `$`: `{$currentUser?.full_name}`. Svelte se suscribe y desuscribe automáticamente.

Tema 4 — Interactividad

Pregunta 19

SELECCIÓN MÚLTIPLE

¿Qué directiva se usa para escuchar un clic en Svelte?

- a `@click`
- b `on:click`
- c `onclick={handler}` ✓
- d `(click)`

Respuesta: c) `onclick={handler}`

Justificación: En Svelte los eventos se escuchan con directivas `on:` como `onclick={handler}`. `Header.svelte` usa `onclick` en el botón de cerrar sesión.

Pregunta 20

SELECCIÓN MÚLTIPLE

¿Qué modificador evita el comportamiento por defecto del envío de un formulario?

- a `onsubmit|stopPropagation`
- b `onsubmit|preventDefault` ✓
- c `onsubmit|self`
- d `onsubmit|once`

Respuesta: b) `onsubmit|preventDefault`

Justificación: El modificador `|preventDefault` llama a `e.preventDefault()`. En formularios (`onsubmit|preventDefault`) evita que la página se recargue.

Pregunta 21

SELECCIÓN MÚLTIPLE

¿Qué función del ciclo de vida se ejecuta una sola vez al montar el componente?

- a `onMount(fn)` ✓
- b `onDestroy(fn)`
- c `$effect(fn)`
- d `onUpdate(fn)`

Respuesta: a) `onMount(fn)`

Justificación: `onMount(fn)` se ejecuta una sola vez al montar el componente. En `+page.svelte` se usa para cargar las calificaciones al iniciar.

Pregunta 22

SELECCIÓN MÚLTIPLE

En Svelte 5, ¿qué reemplaza a las declaraciones reactivas `$:` de Svelte 4?

- a `$state()`
- b `$derived()`
- c `$effect()` ✓
- d `$props()`

Respuesta: c) `$effect()`

Justificación: `$effect()` reacciona a los cambios de sus dependencias y es el reemplazo moderno de las declaraciones reactivas `$:` de Svelte 4.

Pregunta 23

SELECCIÓN MÚLTIPLE

¿Qué directiva permite detectar que el usuario presiona una tecla?

- a `onkeydown={handler}` ✓
- b `oninput={handler}`
- c `onchange={handler}`
- d `onfocus={handler}`

Respuesta: a) `onkeydown={handler}`

Justificación: `onkeydown` detecta teclas. En `+page.svelte` se usa para cerrar el modal con la tecla Escape.

Pregunta 24

SELECCIÓN MÚLTIPLE

¿Qué se debe hacer típicamente en `onDestroy` ?

- a Inicializar el estado global
- b Limpiar recursos: `clearInterval` y `removeEventListener` ✓
- c Renderizar el DOM inicial
- d Hacer la primera petición fetch

Respuesta: b) Limpiar recursos: `clearInterval` y `removeEventListener`

Justificación: `onDestroy` sirve para limpiar recursos (`clearInterval` , `removeEventListener`). En los exámenes se usa para detener el temporizador.

Pregunta 25

SELECCIÓN MÚLTIPLE

¿Qué bloque se usa para renderizado condicional en Svelte?

a `{#if}` ✓

b `{#each}`

c `{#await}`

d `{@html}`

Respuesta: a) `{#if}`

Justificación: `{#if}` renderiza contenido condicionalmente. `Header.svelte` lo usa para mostrar un menú distinto según si el usuario está autenticado.

Pregunta 26

SELECCIÓN MÚLTIPLE

¿Qué bloque se usa para iterar sobre un array en Svelte?

a `{#if}`

b `{#each}` ✓

c `{#loop}`

d `{@each}`

Respuesta: b) `{#each}`

Justificación: `{#each}` itera sobre arrays. El `dashboard` lo usa para listar las calificaciones.

Pregunta 27

SELECCIÓN MÚLTIPLE

¿Cómo se renderiza HTML crudo en Svelte?

a `{@html contenido}` ✓

b `{html}`

c `<div>{{contenido}}</div>`

d `{@render contenido}`

Respuesta: a) `{@html contenido}`

Justificación: `{@html}` renderiza HTML crudo. Debe usarse solo con contenido seguro, porque puede habilitar XSS si el HTML proviene del usuario.

Pregunta 28

SELECCIÓN MÚLTIPLE

En Svelte 5, ¿cómo se renderiza el contenido de un slot dentro de un layout?

- a `<slot />`
- b `{@render children()}` ✓
- c `{@html children}`
- d `{children}`

Respuesta: b) `{@render children()}`

Justificación: En Svelte 5 los slots se renderizan con `{@render children()}`. Reemplaza al `<slot />` de Svelte 4.

Pregunta 29

SELECCIÓN MÚLTIPLE

¿Qué bloque permite manejar el estado de una promesa (cargando/resuelta/error)?

- a `{#if}`
- b `{#each}`
- c `{#await}` ✓
- d `{@promise}`

Respuesta: c) `{#await}`

Justificación: `{#await}` maneja los tres estados de una promesa: pendiente, resuelta (`:then`) y error (`:catch`).

Pregunta 30

SELECCIÓN MÚLTIPLE

En `{#each items as item, i}`, ¿qué representa `i`?

- a El valor del elemento
- b El índice (posición) del elemento ✓
- c El largo del array
- d Un identificador único

Respuesta: b) El índice (posición) del elemento

Justificación: El segundo parámetro de `{#each items as item, i}` es el índice (posición, empezando en 0) del elemento actual.

Pregunta 31

PREGUNTA ABIERTA

Explica la diferencia entre `{#if}` y `{#each}` en Svelte y describe una situación en la que usarías cada uno.

Respuesta modelo: `{#if}` decide si se muestra un bloque según una condición booleana (por ejemplo, mostrar un botón solo si el usuario es admin). `{#each}` repite un bloque por cada elemento de una colección (por ejemplo, listar todas las calificaciones). Suelen combinarse: un `{#each}` para recorrer la lista y un `{#if}` dentro para condicionar cada elemento.

Pregunta 32

PREGUNTA ABIERTA

¿Qué es un slot en un componente de Svelte y cómo se renderiza en Svelte 5? Da un ejemplo breve.

Respuesta modelo: Un slot es un "hueco" que un componente deja para que el componente padre inyecte contenido. En Svelte 5 se renderiza con `{@render children()}`. Ejemplo: un componente `Card` define `<div class="card">{@render children()}</div>` y se usa como `<Card><p>Contenido</p></Card>`.

Tema 6 — Conexión con datos externos

Pregunta 33

SELECCIÓN MÚLTIPLE

¿Qué API nativa de JavaScript se usa para consumir datos de un servidor?

a `fetch()` ✓

b `http()`

c `request()`

d `axios()`

Respuesta: a) `fetch()`

Justificación: `fetch()` es la API nativa para peticiones HTTP. En este proyecto, `src/lib/api.js` envuelve `fetch` para que toda la app hable con el backend.

Pregunta 34

SELECCIÓN MÚLTIPLE

¿Qué header HTTP se usa para enviar un token JWT al servidor?

a Authorization: Bearer <token> ✓

b Token: <token>

c X-Auth: <token>

d Auth: <token>

Respuesta: a) Authorization: Bearer <token>

Justificación: El token JWT se envía en el header `Authorization: Bearer <token>`. En `api.js` se agrega automáticamente leyendo el token de `localStorage`.

Pregunta 35

SELECCIÓN MÚLTIPLE

¿Qué método permite ejecutar varias promesas en paralelo y esperar a todas?

a `Promise.race()`

b `Promise.all()` ✓

c `Promise.any()`

d `Promise.serial()`

Respuesta: b) `Promise.all()`

Justificación: `Promise.all()` ejecuta varias promesas en paralelo y espera a todas. `preloader.ts` lo usa para precargar el perfil y las calificaciones al mismo tiempo.

Pregunta 36

SELECCIÓN MÚLTIPLE

En este proyecto, ¿dónde se guarda el token de autenticación en el navegador?

a `sessionStorage`

b `localStorage` ✓

c Una cookie `httpOnly`

d En la URL

Respuesta: b) `localStorage`

Justificación: En este proyecto el token se guarda en `localStorage` (`auth.js` : `localStorage.setItem("token", ...)`), lo que permite mantener la sesión entre recargas.

Pregunta 37

SELECCIÓN MÚLTIPLE

¿Qué patrón se recomienda para el manejo de errores en operaciones asíncronas?

- a `if/else`
- b `try/catch/finally` ✓
- c `switch/case`
- d `for/while`

Respuesta: b) `try/catch/finally`

Justificación: `try/catch/finally` es el patrón para manejar errores asíncronos. Los formularios de la app lo usan para mostrar el error y, en el `finally`, resetear el estado de carga.

Pregunta 38

SELECCIÓN MÚLTIPLE

¿Qué prefijo deben tener las variables de entorno para quedar expuestas al cliente en Vite?

- a `VITE_` ✓
- b `PUBLIC_`
- c `ENV_`
- d `CLIENT_`

Respuesta: a) `VITE_`

Justificación: En Vite solo las variables con prefijo `VITE_` se exponen al cliente. Por eso la URL del backend se define como `VITE_API_URL`.

Pregunta 39

PREGUNTA ABIERTA

Describe el flujo completo de una petición autenticada: desde el login del usuario hasta la respuesta del servidor (token → request → response).

Respuesta modelo: Flujo completo: 1) `login()` envía usuario/contraseña a `POST /api/auth/login`; 2) el servidor valida con `bcrypt` y devuelve un token JWT; 3) la app guarda el token en `localStorage`; 4) en cada petición, `api.js` agrega `Authorization: Bearer <token>`; 5) el servidor verifica el token (middleware `authenticateToken`) y responde los datos; 6) si el token es inválido o expirado, responde 401/403.

Pregunta 40

PREGUNTA ABIERTA

¿Qué es una API REST y cuáles son sus verbos HTTP principales? Da un ejemplo con Express.js.

Respuesta modelo: Una API REST es un conjunto de endpoints HTTP que exponen recursos siguiendo convenciones. Verbos principales: GET (leer), POST (crear), PUT (actualizar) y DELETE (eliminar). Ejemplo con Express: `app.get("/api/grades", ...)` lista calificaciones y `app.post("/api/grades", ...)` crea una. En este proyecto, `routes/grades.js` define esas rutas y `controllers/gradeController.js` la lógica.

